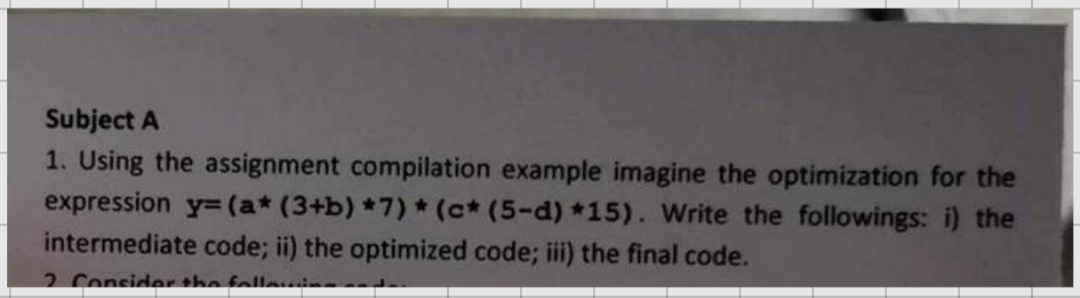
FCPL ex:



Subject A

1. Using the assignment compilation example imagine the optimization for the expression **y= (a* (3+b) *7) * (c* (5-d) *15)**. Write the followings: i) the intermediate code; ii) the optimized code; iii) the final code.

2. Consider the following code:

Course 9 & 10

The goal is to reduce the number of operations at runtime. We use:

1. Constant folding: Compute $7 \times 15 = 105$ at compile time
2. Algebraic Simplification / Reassociation: Group variable computations and Constant computation to minimize temporary variables.

Logic: $(a \cdot (b+3)) \cdot (c \cdot (5-d)) \cdot (7 \cdot 15)$

i) Intermediate Code (Standard Three-Address Code (Course 10)). This is the direct translation before optimization.

$t_1 = 3 + b$
$t_2 = a * t_1$
$t_3 = t_2 * 7$
$t_4 = 5 - d$
$t_5 = c * t_4$
$t_6 = t_5 * 15$
$t_7 = t_3 * t_6$
$y = t_7$

ii) Optimized Code - We apply constant folding and reordering. We calculate

constant 105 and compute the sub-expressions efficiently

$t_1 = b + 3$
$t_2 = 5 - d$
$t_3 = t_1 * t_2$ // $(b+3) * (5-d)$
$t_4 = a * c$
$t_5 = t_3 * t_4$ // combine near parts
$y = t_5 * 105$ // Multiply by folded constant ($7*15$)

iii) Final Code (Target Machine Assembly, Course 11 (Slide 6))

```
LD   R1, b        ;Load b into register 1
ADD  R1, #3       ; R1 = b+3
LD   R2, #5       ; Load 5 into r.2
SUB  R2, d        ; R2 = 5-d
MUL  R1, R2       ; R1 = (b+3)* (5-d)
LD   R2, a        ;  R2 = a
MUL  R2, c        ;  R2 = a*c
MUL  R1, R2       ; R1 = (a*c) *(b+3) * (5-d)
MUL  R1, #105     ; R1 = result *105
ST   y, R1        ; y = final result
```

2. Consider the following code:

```
int tab[3];
void p2(int &x, int &y)
{
   x=y*y;
   y=x*x;
}
void p1(int &x, int &y)
{
   p2(y,x);
   x=y*y;
   y=x*x;
}
void main()
{
   tab[0]=3;
   tab[2]=4;
   p1(tab[0],tab[2]);
}
```

What are the values and how were they obtained for the array elements tab after the call to procedure p1 if parameter transmission is made by reference.

3. Write a type in C containing an array of pointers which

Curs 8

(RUNTIME ENV, Slide 22 - Call by Reference)

Concept: In Call by Reference, the L-value (address) of the actual parameter is passed. The formal parameter becomes an alias for the actual parameter.

Exec trace

Main:

tab[0] = 3 , tab[2] = 4

Call p_1 (tab[0], tab[2])

1. Inside P_1:

Call p_2 (y, x)

p_2 local x refrences p_1's y → tab[2]

p_2 local y — 1 — p_1's x → tab[0]

Inside P_2:

- x = y * y → tab[2] = tab[0] * tab[0] → tab[2] = 3*3 = 9
- y = x * x → tab[0] = tab[2] * tab[2] → tab[0] = 9 * 9 = 81

(End of p2 . Current state: tab[0] = 81 , tab[2] = 9)

2. Back in p_1:

- x = y * y (Remember: x is tab[0], y is tab[2])
- tab[0] = tab[2] * tab[2] → tab[0] = 9 * 9 = 81
- y = x * x;
- tab[2] = tab[0] * tab[0] → tab[2] = 81 * 81 = 6561

Final Result:
- tab[0] = 81
- tab[2] = 6561

3. Write a type in C containing an array of pointers which point to the type itself and a program that creates a 3 level tree having the degree of 2 using that type.
4. Write the operational sematic for the instruction if

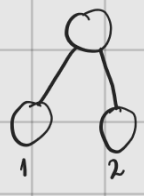
Course 7
(Type Checking)

Type Definition

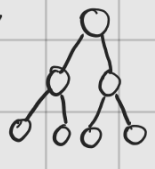
```
typedef struct Node {
        int value;                      // data payload
        struct Node * child[2];         // Array of pointers to type itself ( 2n degree )
} Node;
```

Program (3 Level Tree)

degree of 2 - 2 children



level 3 - depth

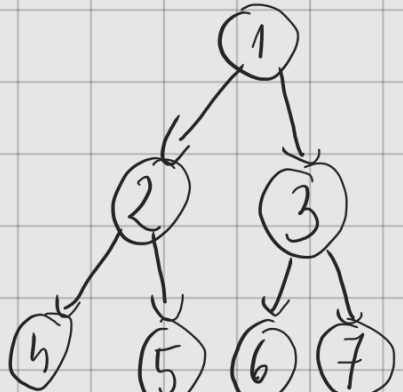


```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
        int value;
        struct Node * child[2];
} Node;

Node* new_node (int v) {
        Node *n = (Node*) malloc (sizeof(Node));
        n->value = v
        n->child[0] = Null;
        n->child[1] = Null;
        return n;
}

int main() {
        // Level 1 (root)
```



```
Node *root = nene_node(1);

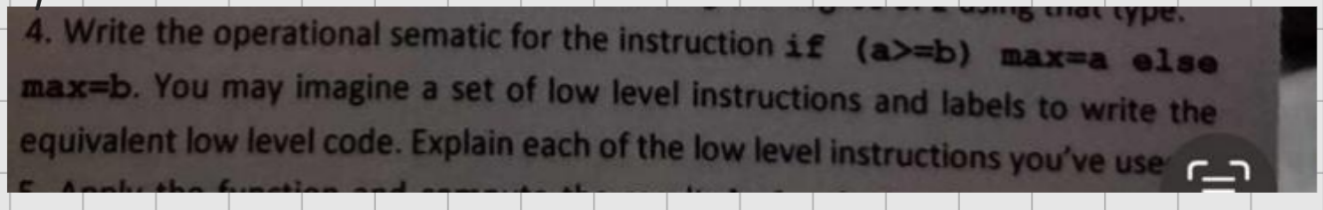
// Level 2

root -> child[0] = nene_node(2);
root -> child[1] = nene_node(3);

// Level 3

root -> child[0] -> child[0] = nene_node(4);
root -> child[0] -> child[1] = nene_node(5);
root -> child[1] -> child[0] = nene_node(6);
root -> child[1] -> child[1] = nene_node(7);

return 0;
}
```



4. Semantics (Course 5) - slide 10 - operational semantics
Low Level (Course 10) - slide 20 - Control Flow.

i) Operational Semantics, We use the configuration notation
$\langle$ Statement, σ $\rangle \rightarrow \sigma'$ where σ is the state mem.

Case 1: (True cond.)

$$\frac{\langle a \geq b, \sigma \rangle \rightarrow \text{true} \quad \langle max = a, \sigma \rangle \rightarrow \sigma'}{\langle \text{if } (a \geq b) \; max = a \text{ else } max = b, \sigma \rangle \rightarrow \sigma'}$$

(Where $\sigma' = \sigma[max \mapsto \sigma(a)]$)

Case 2 : (False)

$$\frac{\langle a \geq b, \sigma \rangle \to false \quad \langle max = b, \sigma \rangle \to \sigma''}{\langle \text{if } (a \geq b) max = a \text{ else } max = b, \sigma \rangle \to \sigma''}$$

(Where $\sigma'' = \sigma[max \mapsto \sigma(b)]$)

ii) Low Level Code (Course 10 Three-address Code with Jumps):

1. if $a < b$ goto L_1
2. max = a
3. goto L_2
4. L_1 : max = b
5. L_2 : ....

Explanation of Instructions:

1. `if a < b goto L1` : A **Conditional Jump**. It compares `a` and `b`. If the condition ($a < b$) is true, the program counter jumps to label `L1`. This implements the `else` logic (since if $a < b$, $a \geq b$ is false).
2. `max = a` : An **Assignment**. Executed only if the jump in step 1 was *not* taken (meaning $a \geq b$).
3. `goto L2` : An **Unconditional Jump**. After setting max to `a`, we must skip the `else` block and jump to the end of the structure (`L2`).
4. `L1: max = b` : A **Labelled Assignment**. This is the target of the jump from step 1. It sets max to `b`.
5. `L2:` : A **Label**. Marks the end of the if-else structure.

- σ (Sigma) - current state of the computer memory. It's

a mapping from variables to values. -ex: $\sigma(a) = 10$, means var a holds value 10.

- $\langle Code, \sigma \rangle$ - "we are about to execute this code in the current memory state σ."
- $\longrightarrow$ transition arrow (means: "evaluates to / result in new state")
- $\sigma[x \mapsto v]$: This is an update operation. "Old state σ, but updated so that variable x now holds value v"

Semantic Rule: Top part: condition (what must be true, for this rule to apply) Bottom part: result (what happens)

5. Apply the function and compute the result: λe1. λe2. λc. (c e1 e
ex1 ex2

Course 3 (Slide 15 - Beta Reduction)

Expression: $(\lambda e1.\ \lambda e2.\ \lambda c.\ (c\ e1\ e2))\ ex1\ ex2$

We perform Beta Reduction (application): $(\lambda x.M)N \rightarrow_\beta M[x := N]$

1. Apply ex_1 to e_1:

$$(\lambda e_1.(\lambda e_2.\lambda c(c\ e_1\ e_2)))\ ex_1 \rightarrow \lambda e_2.\lambda c(c\ ex_1\ e_2)$$

(Replace all e_1 with ex_1) + λe_1 (BYE BYE)

2. Apply ex_2 to e_2:

$$(\lambda e_2.(\lambda c.(c\ ex_1\ e_2)))\ ex_2 \rightarrow \lambda c(c\ ex_1\ ex_2)$$

(Replace all e_2 with ex_2) + λe_2 (BYE BYE)

Final result: Only c remains

$\lambda c.(c\ ex_1\ ex_2)$ → ex_1, ex_2 are now stored together and are waiting for a selector / consumer (c).

(This function waits for a third argument c (a selector) to choose between ex_1 and ex_2. This is church encoding for a Pair)

BETA REDUCTION: - function application.

(Plugging arguments into a function).

The rule:

1. Take the first argument outside the function
2. Find the very first parameter name defined in the function header
3. Scan the BODY and replace every occurance of that) (right after λ)

parameter name with the argument.

4. Remove the initial λ header.
5. Repeat until you run out of arguments.

Crucial: In Lambda Calculus, functions accept only one argument at a time.

Interpretation (What does this mean?)

In theoretical computer science, this is a very famous structure known as a **Church Pair**.

In a functional language that only has functions (no built-in data structures like arrays or structs), how do you store two values together? You create a function that "holds onto" them.

The result $\lambda c.(c\ ex1\ ex2)$ is a function that is holding `ex1` and `ex2` captive. It is waiting for one final argument, `c` (often called a "selector" or "consumer" function). Once you give it a `c`, it will execute `c` on the two values it is holding.

- The original function you started with was a **Pair Constructor**.
- You gave it two values, and it returned a **Pair** holding those values.